Data Ingestion

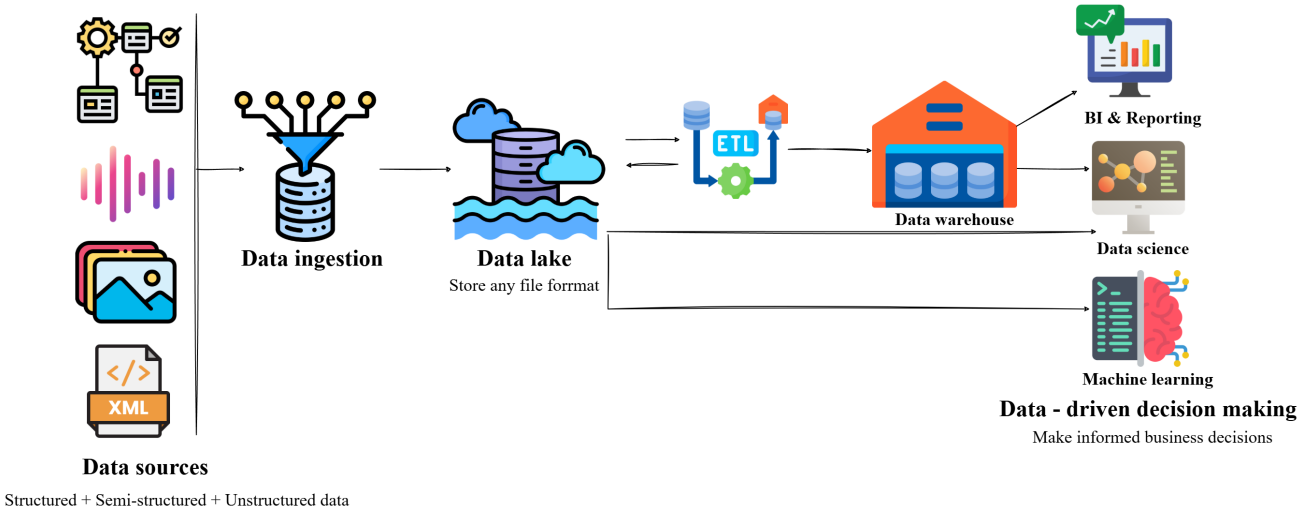
Tạ Hữu Anh Dương **Dương Nguyễn Thuận**

Proofreading: Đinh Quang Vinh

I. Bối cảnh

Trong các hệ thống học máy và trí tuệ nhân tạo hiện đại, chất lượng của mô hình phụ thuộc trực tiếp vào chất lượng của dữ liệu đưa vào. Một mô hình dù có kiến trúc tốt đến đâu cũng khó cho kết quả đáng tin cậy nếu dữ liệu huấn luyện nghèo nàn, thiếu nhất quán hoặc không được cập nhật. Vì vậy, trước khi nói tới mô hình, ta cần hiểu dữ liệu đi từ đâu tới và bằng cách nào nó được đưa vào hệ thống.

Trên thực tế, dữ liệu hiếm khi nằm sẵn ở dạng có thể dùng ngay. Nó phát sinh rải rác từ nhiều nơi: cơ sở dữ liệu của doanh nghiệp, các API dịch vụ web, tệp tin, thiết bị cảm biến hay mạng xã hội. Mỗi nguồn lại có định dạng và mức độ tổ chức khác nhau. Để biến những mảnh dữ liệu rời rạc thành một nguồn tài nguyên thống nhất phục vụ cho quá trình xử lí và phân tích, dữ liệu thường trải qua ba bước chính: **kết nối tới các nguồn dữ liệu** (data sources), **thu thập và chuyển dữ liệu** từ các nguồn vào hệ thống lưu trữ trung tâm (data ingestion), rồi **lưu trữ dữ liệu** trong một hệ thống phù hợp. Trong phạm vi chương này, hệ thống lưu trữ được sử dụng là Data Lake, cho phép lưu trữ linh hoạt nhiều loại dữ liệu thô với định dạng khác nhau.



Hình 1: Bức tranh tổng thể của một kiến trúc dữ liệu hiện đại: dữ liệu thô từ nhiều nguồn (có cấu trúc, bán cấu trúc và phi cấu trúc) được thu thập và nạp vào data lake, sau đó xử lý qua ETL rồi đưa vào data warehouse để phục vụ BI, khoa học dữ liệu và học máy, hướng tới việc ra quyết định dựa trên dữ liệu.

Tài liệu này đi theo đúng hành trình đó. Đầu tiên, phần Data Source giới thiệu các nhóm nguồn dữ liệu và cách phân loại chúng theo nguồn gốc cũng như theo mức độ cấu trúc. Tiếp theo, phần Data Ingestion trình bày quá trình đưa dữ liệu từ nguồn về hệ thống, cùng ba phương pháp nạp phổ biến là batch, micro-batching và streaming. Sau đó, phần Data Lake nói về nơi lưu trữ tập trung dành cho dữ liệu thô và một vài nền tảng tiêu biểu. Cuối cùng, phần Thực hành đúc kết toàn bộ kiến thức trên thành một pipeline cụ thể, tự động thu thập dữ liệu mỗi ngày từ ba nguồn thực tế là YouTube, Unsplash và arXiv rồi ghi vào một Data Lake thu nhỏ trên máy.

Mục lục

I. Bối cảnh **1**

II. Data Source **4**

II.1. Phân loại theo nguồn gốc 5

II.2. Phân loại theo cấu trúc 6

III. Data Ingestion **7**

III.1. Giới thiệu về Data ingestion 7

III.2. Các phương pháp Data Ingestion 8

IV. Data Lake **10**

IV.1. Giới thiệu về Data lake 10

IV.2. Một số Data Lake tiêu biểu 11

V. Thực hành **13**

V.1. Giới thiệu bài thực hành 13

V.1.1. Cấu trúc bài thực hành 14

V.2. Giới thiệu các nguồn dữ liệu 15

V.2.1. YouTube 15

V.2.2. Unsplash 15

V.2.3. arXiv 15

V.3. Triển khai bài thực hành 16

V.3.1. Lấy dữ liệu từ YouTube (Video + Audio + Phụ đề) 16

V.3.2. Lấy dữ liệu từ Unsplash 18

V.3.3. Lấy dữ liệu từ arXiv API 20

V.4. Tạo lịch lấy dữ liệu hàng ngày 22

V.4.1. Điều phối pipeline với `run_ingestion.py` 22

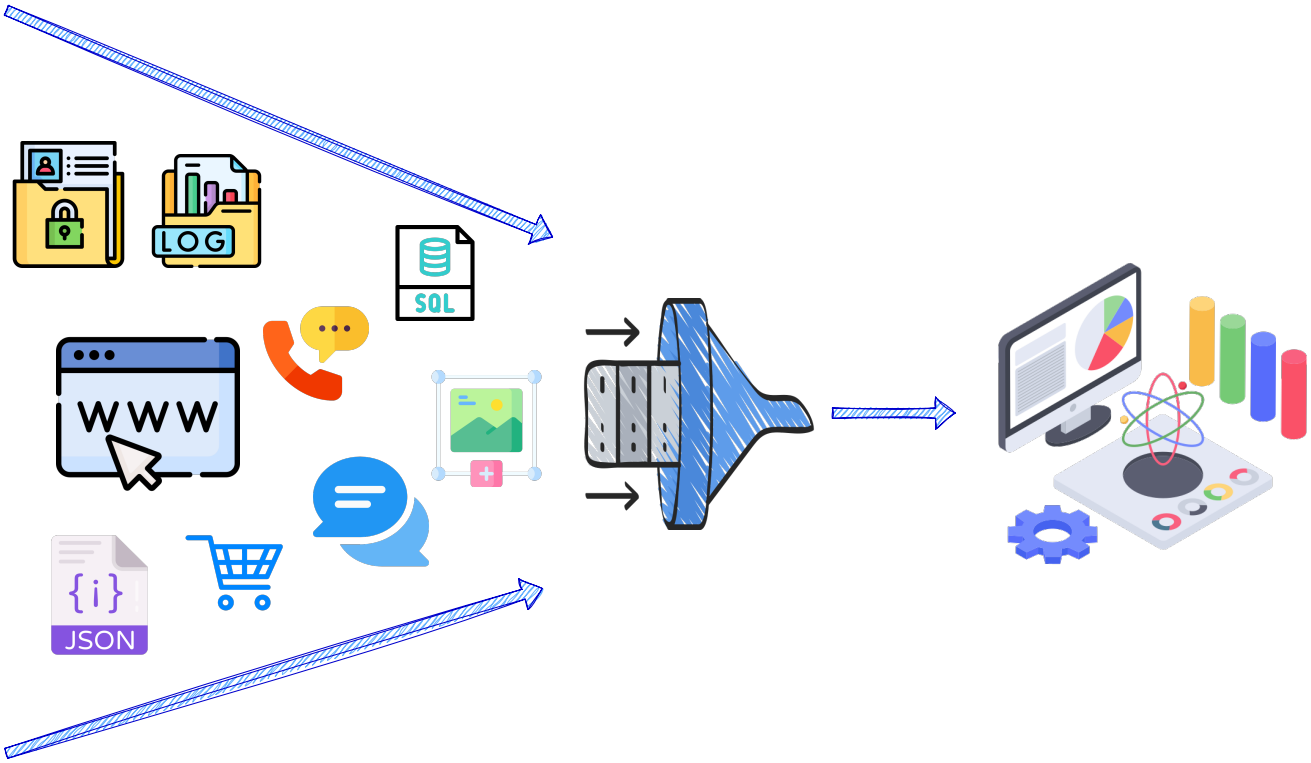
V.4.2. Đăng ký Windows Task Scheduler 23

V.4.3. Kết luận phần thực hành 23

VI. Tài liệu tham khảo **24**

II. Data Source

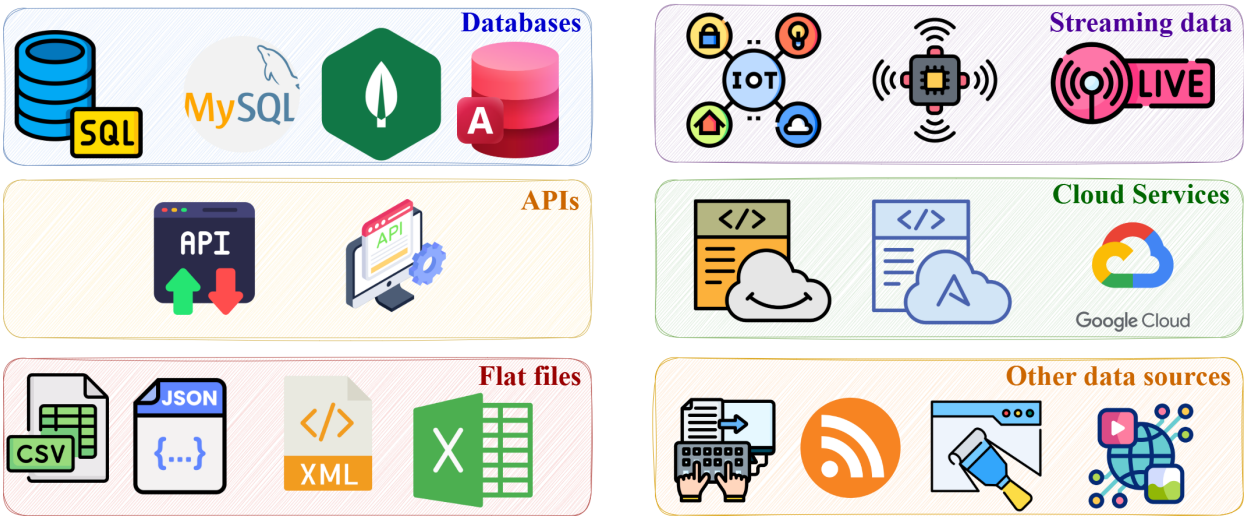
Data source là nguồn cung cấp dữ liệu cho một hệ thống. Nguồn dữ liệu có thể lưu trữ, quản lý hoặc tạo ra dữ liệu, đồng thời cho phép dữ liệu được truy cập hoặc truyền tới các hệ thống khác để phục vụ việc thu thập, xử lý, phân tích và khai thác.



Hình 2: Data source là điểm khởi đầu của dòng chảy dữ liệu. Đây là nơi dữ liệu được tạo ra hoặc lưu trữ, chẳng hạn như ứng dụng web, cơ sở dữ liệu, tệp CSV hoặc JSON, hình ảnh, video, cảm biến IoT và các hệ thống nhắn tin. Từ các nguồn này, dữ liệu sẽ được thu thập để đưa vào hệ thống xử lý.

Data sources có thể được phân loại theo hai tiêu chí bổ trợ cho nhau. Thứ nhất, phân loại theo nguồn gốc và phương thức cung cấp dữ liệu nhằm xác định dữ liệu phát sinh từ đâu và được đưa vào hệ thống như thế nào. Thứ hai, phân loại theo mức độ cấu trúc nhằm xác định dữ liệu được tổ chức chặt chẽ đến đâu, qua đó quyết định công cụ và phương pháp phân tích phù hợp. Một nguồn dữ liệu cụ thể đồng thời thuộc về một nhóm theo nguồn gốc và mang một dạng cấu trúc nhất định.

II.1. Phân loại theo nguồn gốc



Hình 3: Các nhóm nguồn dữ liệu phân loại theo nguồn gốc và định dạng

1. **Databases:** Cơ sở dữ liệu là nguồn dữ liệu truyền thống và phổ biến nhất, lưu trữ dữ liệu có cấu trúc trong các hệ quản trị cơ sở dữ liệu quan hệ hoặc phi quan hệ. Các hệ thống xử lý giao dịch trực tuyến (Online Transaction Processing – OLTP), chẳng hạn như CRM, ERP và HRM, ghi nhận các hoạt động hằng ngày của doanh nghiệp. Cơ sở dữ liệu của các hệ thống này được tối ưu cho việc xử lý giao dịch nhanh và đảm bảo tính nhất quán của dữ liệu, thay vì thực hiện các truy vấn phân tích trên khối lượng dữ liệu lớn. Do dữ liệu thường được chuẩn hóa nhằm giảm dư thừa và đảm bảo tính toàn vẹn, việc thực hiện các truy vấn phân tích trực tiếp có thể làm giảm hiệu năng của hệ thống. Vì vậy, dữ liệu thường được trích xuất thông qua quy trình ETL và nạp vào data warehouse để phục vụ các tác vụ phân tích và báo cáo.

2. **APIs:** Application Programming Interface đóng vai trò cầu nối cho phép các phần mềm giao tiếp và trao đổi dữ liệu với nhau. Thông qua API, hệ thống có thể truy cập dữ liệu theo yêu cầu (on-demand) hoặc gần thời gian thực từ các dịch vụ web, nền tảng đám mây và ứng dụng của bên thứ ba. Phương thức này đặc biệt phổ biến khi thu thập dữ liệu từ mạng xã hội, cổng thanh toán hoặc dịch vụ phân tích bên ngoài.

3. **Flat files:** Là các tệp văn bản đơn giản lưu dữ liệu ở định dạng có cấu trúc hoặc bán cấu trúc, tiêu biểu như CSV, TSV, TXT hoặc bảng tính Excel; ngoài ra nhiều hệ thống cũng sử dụng JSON và XML để trao đổi dữ liệu bán cấu trúc. Nhờ tính gọn nhẹ và dễ trao đổi, tệp phẳng thường được dùng để xuất/nhập dữ liệu giữa các hệ thống hoặc làm bước trung gian trong quy trình ETL. Tuy nhiên, việc quản lý và phân tích trở nên khó khăn khi kích thước tệp lớn.

4. **Streaming data:** Là dữ liệu được tạo ra và truyền liên tục dưới dạng các sự kiện (events), thường thông qua các nền tảng như Kafka, RabbitMQ hoặc MQTT. Nguồn phát sinh có thể là cảm biến IoT, hệ thống giao dịch, nhật ký hệ thống hoặc các ứng dụng trực tuyến.

Loại dữ liệu này thiết yếu cho những ứng dụng yêu cầu phản hồi tức thời như giám sát an ninh mạng hay theo dõi sự kiện trực tiếp.

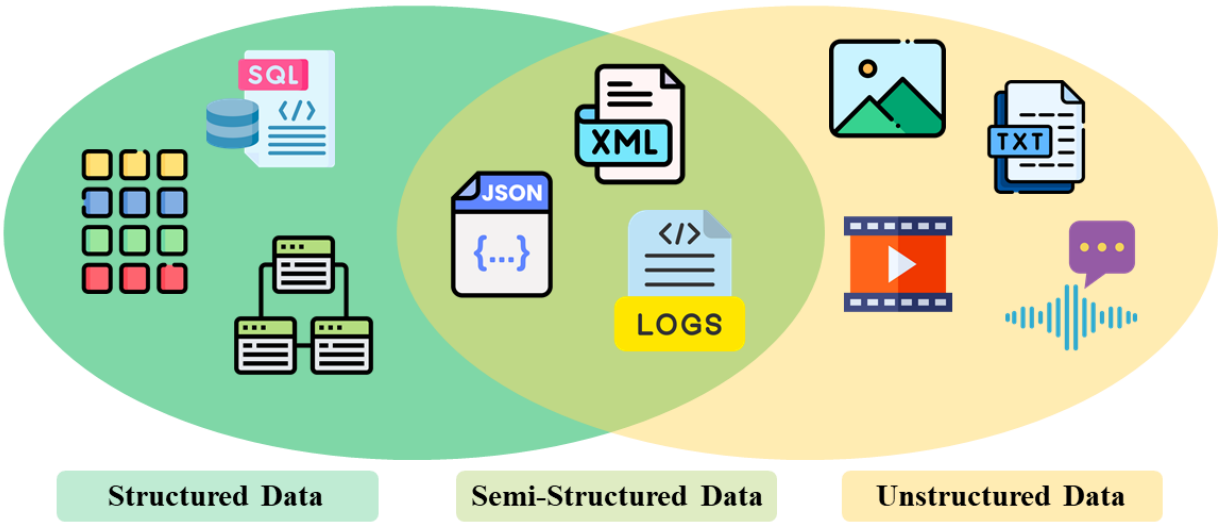
5. **Cloud services:** Các dịch vụ lưu trữ và cơ sở dữ liệu được quản lý trên nền tảng đám mây, chẳng hạn như Amazon S3, Google Cloud Storage, Azure Blob Storage hoặc BigQuery, là những nguồn dữ liệu phổ biến trong các hệ thống hiện đại. Chúng cung cấp khả năng mở rộng linh hoạt, độ sẵn sàng cao và dễ dàng tích hợp với các quy trình phân tích dữ liệu.

6. **Other data sources:** Nhóm này gồm các nguồn dữ liệu như RSS feed, website, diễn đàn và mạng xã hội. Dữ liệu từ các nguồn này thường được thu thập thông qua API hoặc web scraping và chủ yếu ở dạng phi cấu trúc hoặc bán cấu trúc.

7. **Application and system logs:** Nhật ký hệ thống và ứng dụng ghi lại các sự kiện xảy ra trong quá trình vận hành, chẳng hạn như truy cập của người dùng, lỗi hệ thống hoặc hoạt động của máy chủ. Đây là nguồn dữ liệu quan trọng trong giám sát hệ thống, bảo mật và phân tích hành vi người dùng.

8. **IoT and sensor data:** Các cảm biến, thiết bị IoT, GPS và camera liên tục tạo ra dữ liệu về môi trường hoặc trạng thái của thiết bị. Dữ liệu này thường được truyền theo thời gian thực và được sử dụng trong các hệ thống giám sát, sản xuất thông minh và thành phố thông minh.

II.2. Phân loại theo cấu trúc



Hình 4: Các dạng nguồn dữ liệu phân loại theo mức độ cấu trúc

Nếu tiêu chí nguồn gốc trả lời cho câu hỏi dữ liệu đến từ đâu, thì tiêu chí cấu trúc trả lời cho câu hỏi dữ liệu được tổ chức và biểu diễn như thế nào. Dựa trên mức độ tổ chức, mọi nguồn dữ liệu đều có thể quy về một trong ba dạng sau:

1. **Structured Data Sources:** Dữ liệu có cấu trúc là dữ liệu được tổ chức theo một lược đồ (schema) xác định trước, thường ở dạng bảng gồm các hàng và cột. Nhờ cấu trúc rõ ràng,

dữ liệu có thể được lưu trữ, truy vấn và phân tích một cách hiệu quả bằng các hệ quản trị cơ sở dữ liệu. Nguồn dữ liệu tiêu biểu gồm cơ sở dữ liệu quan hệ (MySQL, PostgreSQL, SQL Server), bảng tính Excel và các tệp CSV có cấu trúc. Loại dữ liệu này thường lưu trữ dữ liệu giao dịch, dữ liệu khách hàng, dữ liệu tài chính và các thông tin phục vụ vận hành hằng ngày của doanh nghiệp.

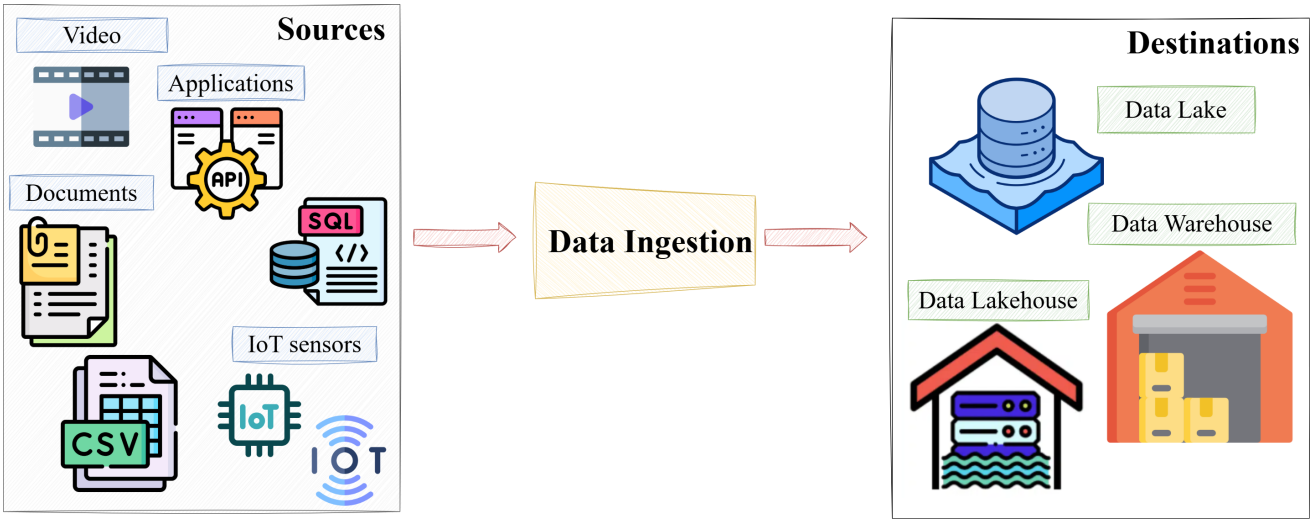
2. **Unstructured Data:** Dữ liệu phi cấu trúc là dữ liệu không có lược đồ (schema) cố định hoặc không thể biểu diễn trực tiếp dưới dạng bảng gồm hàng và cột. Vì vậy, việc lưu trữ, tìm kiếm và phân tích thường phức tạp hơn so với dữ liệu có cấu trúc. Các ví dụ phổ biến gồm văn bản, thư điện tử, bài đăng mạng xã hội, hình ảnh, âm thanh và video. Loại dữ liệu này thường được lưu trữ trong Data Lake hoặc các hệ thống lưu trữ đối tượng (Object Storage) trước khi được xử lý bằng các kỹ thuật như xử lý ngôn ngữ tự nhiên (NLP), thị giác máy tính (Computer Vision) hoặc học máy.

3. **Semi-Structured Data:** Dữ liệu bán cấu trúc là dạng dữ liệu không tuân theo một lược đồ cố định như dữ liệu có cấu trúc, nhưng vẫn chứa các thành phần mô tả (metadata) dưới dạng khóa–giá trị (key–value) hoặc thẻ đánh dấu (tags), giúp việc phân tích và xử lý thuận lợi hơn. Các định dạng phổ biến gồm JSON, XML, HTML và YAML. Nhờ tính linh hoạt, dữ liệu bán cấu trúc thường được sử dụng để trao đổi thông tin giữa các hệ thống và có thể được phân tích cú pháp (parse) để chuyển đổi thành dữ liệu có cấu trúc trước khi lưu vào cơ sở dữ liệu hoặc Data Warehouse.

III. Data Ingestion

III.1. Giới thiệu về Data ingestion

Data ingestion là quá trình thu thập và đưa dữ liệu từ nhiều nguồn khác nhau vào một hệ thống lưu trữ hoặc xử lý trung tâm. Các nguồn dữ liệu có thể bao gồm cơ sở dữ liệu, ứng dụng SaaS, tệp dữ liệu, API, thiết bị IoT hoặc luồng dữ liệu thời gian thực. Tùy theo kiến trúc của hệ thống, dữ liệu sau khi được thu thập có thể được đưa vào Data Lake, Data Warehouse, Lakehouse hoặc các nền tảng xử lý dữ liệu khác.



Hình 5: Tổng quan quá trình data ingestion: thu thập dữ liệu từ nhiều nguồn và đưa về kho lưu trữ tập trung.

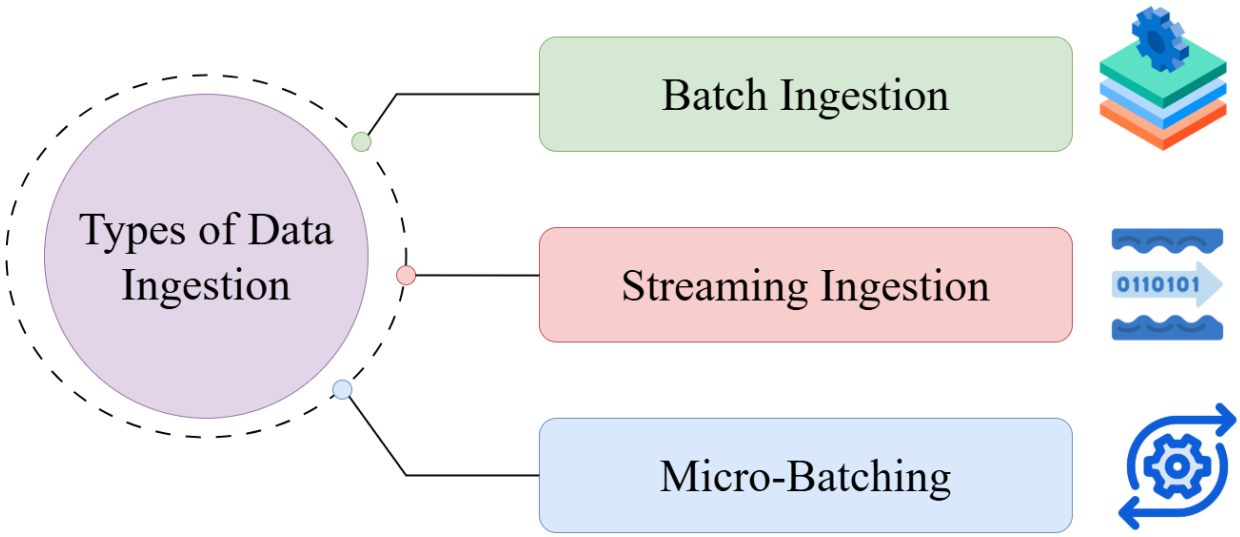
Lưu ý

Data ingestion và ETL

- Khác với quy trình ETL phải xử lý và biến đổi dữ liệu trước khi lưu, data ingestion đẩy dữ liệu thô thẳng vào hệ thống đích. Nhờ bỏ qua bước biến đổi trung gian, việc đưa dữ liệu vào hệ thống diễn ra nhanh và linh hoạt hơn, đổi lại dữ liệu thường cần được làm sạch và chuẩn hóa ở các bước sau.

III.2. Các phương pháp Data Ingestion

Tùy theo yêu cầu về độ trễ và tần suất cập nhật, data ingestion được triển khai theo ba phương pháp chính: nạp theo lô (batch), micro-batching và nạp theo luồng (streaming), được minh họa trong Hình 6.



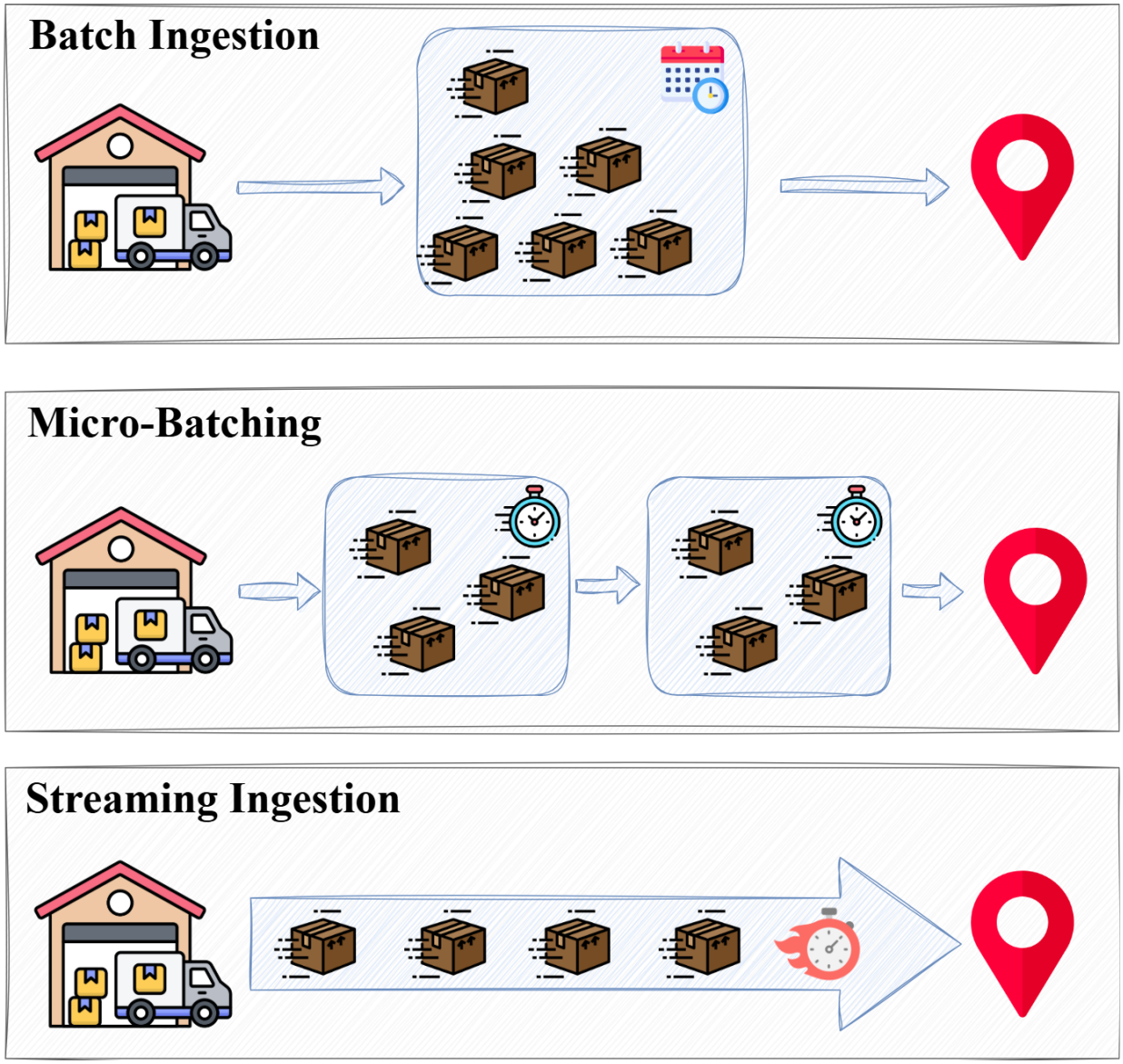
Hình 6: Ba phương pháp nạp dữ liệu: batch, micro-batching và streaming.

1. **Batch ingestion:** Nạp dữ liệu theo lô di chuyển dữ liệu thành từng đợt tại các khoảng thời gian định trước, có thể tính bằng phút, giờ, ngày hoặc tuần. Phương pháp này cho phép đội ngũ phát triển và phân tích thu thập dữ liệu theo lịch trình hoặc theo sự kiện kích hoạt, ví dụ thu thập một tập dữ liệu cụ thể từ nền tảng CRM vào cùng một ngày mỗi tháng. Cách tiếp cận này phù hợp với dữ liệu không đòi hỏi xử lý theo thời gian thực, đồng thời được dùng để thu thập định kỳ các điểm dữ liệu phục vụ phân tích chuyên sâu.

2. **Micro-batching:** Phương pháp này dung hòa giữa tốc độ cập nhật nhanh và khả năng xử lý lượng lớn dữ liệu bằng cách thực thi các lô nhỏ gọn sau mỗi vài mili giây hoặc vài giây. Nhờ đó, nó khắc phục được nhược điểm chờ đợi lâu của nạp lô truyền thống. Một ví dụ điển hình là các dịch vụ giám sát hành trình áp dụng micro-batching để làm mới trạng thái xe mỗi 1 đến 2 giây.

3. **Streaming ingestion:** Nạp dữ liệu theo luồng, hay nạp theo thời gian thực, sử dụng các cơ chế như thu thập dữ liệu thay đổi (Change Data Capture, CDC) để di chuyển dữ liệu từ nguồn ngay khi phát sinh. Mỗi khi dữ liệu nguồn thay đổi, hệ thống đồng bộ các thay đổi này mà không làm gián đoạn khối lượng công việc hiện tại của cơ sở dữ liệu. Phương pháp này thiết yếu cho các tình huống nhạy cảm về thời gian, khi doanh nghiệp phải phản ứng tức thời với thông tin mới, chẳng hạn giao dịch trên thị trường chứng khoán. Ngoài ra, nó còn hỗ trợ ra quyết định vận hành thông qua các pipeline nhanh để phát hiện thông tin chuyên sâu.

Hình 7 minh họa một cách trực quan sự khác biệt về dòng chảy dữ liệu giữa ba phương pháp, từ các lô lớn theo lịch của batch, các lô nhỏ liên tiếp của micro-batching, cho đến dòng dữ liệu liên tục của streaming.

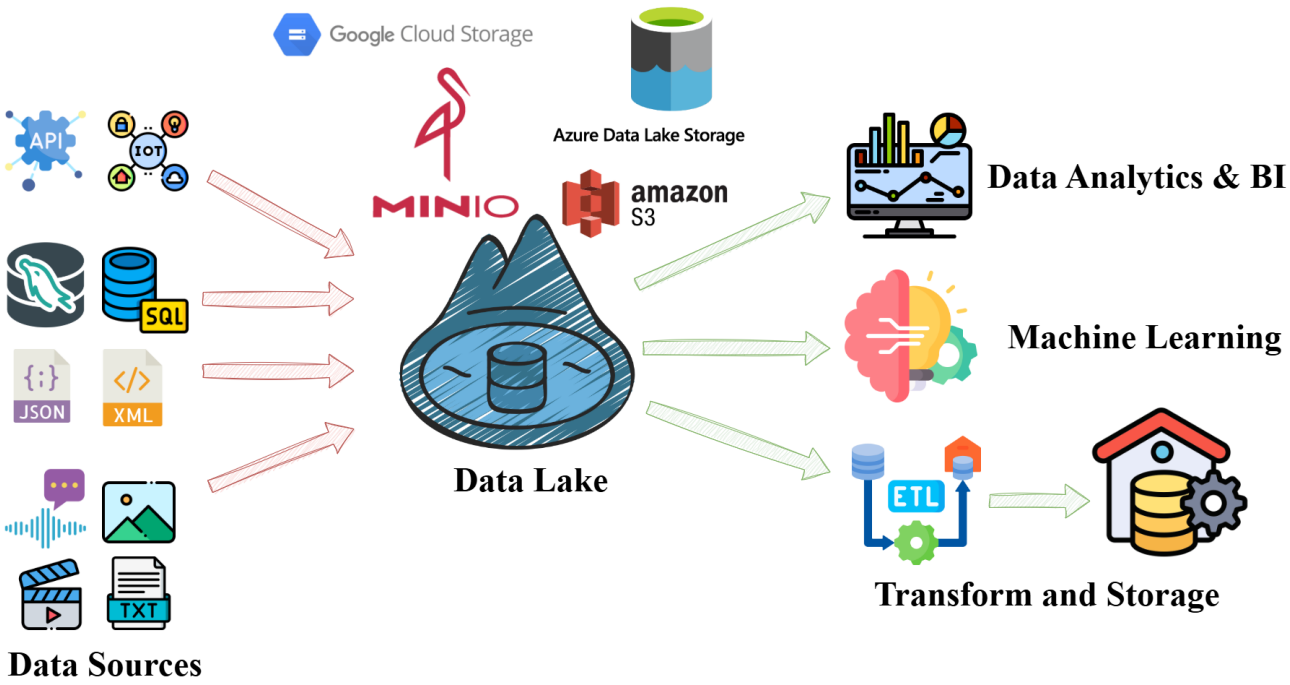


Hình 7: Minh họa trực quan ba phương pháp nạp dữ liệu. Batch gom dữ liệu thành từng lô lớn theo lịch trình; micro-batching chia thành nhiều lô nhỏ liên tiếp; streaming đẩy dữ liệu thành dòng liên tục ngay khi phát sinh.

IV. Data Lake

IV.1. Giới thiệu về Data lake

Data lake là một kho lưu trữ tập trung được thiết kế để lưu trữ khối lượng lớn dữ liệu ở định dạng nguyên bản (raw format). Trong các hệ thống hiện đại, data lake thường được xây dựng trên các nền tảng lưu trữ đối tượng (object storage) như Amazon S3, Azure Blob Storage hoặc Google Cloud Storage nhờ khả năng mở rộng linh hoạt và chi phí lưu trữ thấp. Nhờ không yêu cầu định nghĩa lược đồ trước khi ghi dữ liệu, data lake cho phép thu thập và lưu trữ đồng thời dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc trong cùng một nền tảng.



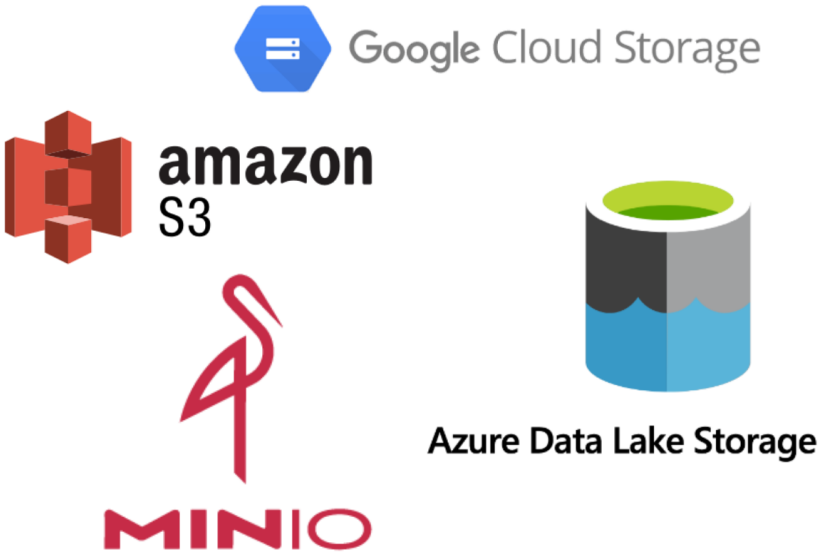
Hình 8: Data Lake đóng vai trò kho lưu trữ trung tâm, lưu trữ dữ liệu được thu thập từ nhiều nguồn với nhiều định dạng khác nhau (API, cơ sở dữ liệu, JSON/XML, hình ảnh, âm thanh). Data Lake thường được triển khai trên các nền tảng lưu trữ đối tượng như Amazon S3, Google Cloud Storage và Azure Data Lake Storage, đồng thời cung cấp dữ liệu cho các hoạt động phân tích, Business Intelligence (BI), học máy và các hệ thống xử lý dữ liệu tiếp theo.

Ngày nay, Data Lake đã trở thành một thành phần cốt lõi trong kiến trúc dữ liệu của nhiều tổ chức. Data Lake có thể đảm nhận nhiều vai trò khác nhau: làm kho lưu trữ tập trung với chi phí thấp và khả năng mở rộng cao; lưu trữ dữ liệu lịch sử hoặc dữ liệu ít được truy cập; đóng vai trò khu vực tiếp nhận (landing zone) cho dữ liệu mới trước khi xử lý; đồng thời lưu trữ các tập dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc phục vụ cho khoa học dữ liệu, học máy, trí tuệ nhân tạo và phân tích dữ liệu lớn.

Tuy nhiên, nếu thiếu các cơ chế quản trị dữ liệu (data governance) và lập danh mục dữ liệu (data catalog) phù hợp, Data Lake có thể trở thành một "Data Swamp"– nơi dữ liệu được lưu trữ với khối lượng lớn nhưng khó tìm kiếm, quản lý và khai thác hiệu quả.

IV.2. Một số Data Lake tiêu biểu

Trên thực tế, phần lớn data lake được triển khai trên hạ tầng lưu trữ đối tượng của các nhà cung cấp đám mây lớn, kết hợp cùng các lớp quản trị và định dạng bảng mở (open table format). Một số nền tảng tiêu biểu có thể kể đến như:



Hình 9: Một số nền tảng lưu trữ phổ biến dùng để xây dựng data lake: Amazon S3, Google Cloud Storage, Azure Data Lake Storage và MinIO.

- **Amazon S3 (Simple Storage Service):** Dịch vụ lưu trữ đối tượng của AWS, được xem là nền tảng phổ biến nhất để xây dựng data lake nhờ độ bền cao, khả năng mở rộng gần như không giới hạn và chi phí thấp. S3 thường được kết hợp với AWS Lake Formation để quản trị quyền truy cập, cùng các dịch vụ như AWS Glue và Amazon Athena để lập danh mục và truy vấn dữ liệu trực tiếp trên data lake.
- **Google Cloud Storage (GCS):** Dịch vụ lưu trữ đối tượng của Google Cloud, đóng vai trò lớp lưu trữ trung tâm cho data lake trên nền tảng này. GCS tích hợp chặt chẽ với hệ sinh thái phân tích của Google như BigQuery và Dataproc, phù hợp cho các khối lượng công việc phân tích và học máy quy mô lớn.
- **Azure Data Lake Storage (ADLS):** Giải pháp lưu trữ data lake của Microsoft Azure, được xây dựng trên nền Azure Blob Storage và bổ sung không gian tên phân cấp (hierarchical namespace) giúp tổ chức dữ liệu theo thư mục hiệu quả hơn. ADLS tích hợp tốt với các công cụ như Azure Synapse Analytics và Databricks, hướng tới các kịch bản phân tích dữ liệu lớn trong doanh nghiệp.
- **MinIO:** Giải pháp lưu trữ đối tượng mã nguồn mở, tương thích với giao diện API của Amazon S3. MinIO cho phép tổ chức tự triển khai data lake trên hạ tầng riêng (on-premises) hoặc môi trường đám mây lai (hybrid cloud), phù hợp với các trường hợp cần kiểm soát dữ liệu chặt chẽ hoặc muốn tránh phụ thuộc vào một nhà cung cấp đám mây duy nhất.

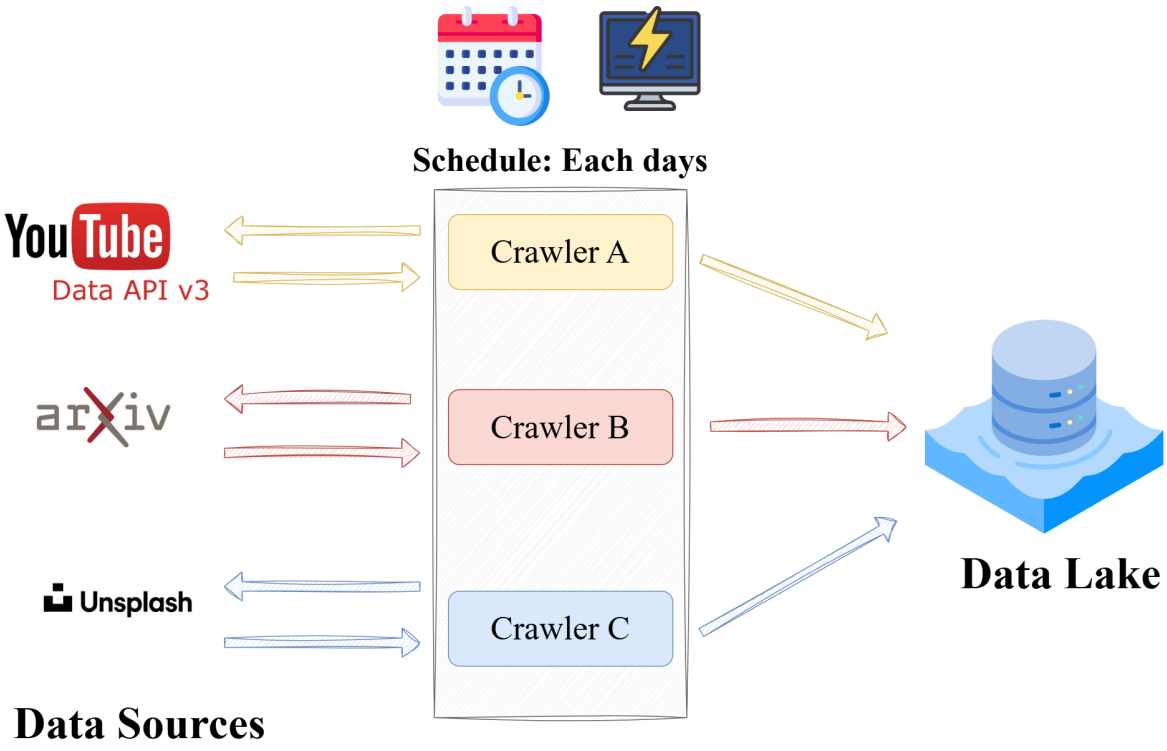
Điểm chung của các nền tảng trên là đều dựa trên mô hình lưu trữ đối tượng, cho phép lưu trữ dữ liệu ở mọi định dạng với chi phí thấp và khả năng mở rộng cao.

V. Thực hành

V.1. Giới thiệu bài thực hành

Phần thực hành này minh họa quy trình **nạp dữ liệu** (data ingestion), một trong những bước nền tảng của hệ thống dữ liệu. Cụ thể, chúng ta sẽ xây dựng một pipeline tự động thu thập dữ liệu mới mỗi ngày từ ba nguồn bên ngoài là **YouTube**, **Unsplash** và **arXiv**, sau đó lưu về máy theo cấu trúc thư mục rõ ràng.

Trong ba phương pháp nạp dữ liệu đã trình bày ở phần lý thuyết, pipeline này đi theo hướng batch ingestion: thay vì theo dõi nguồn liên tục, hệ thống chạy theo lịch cố định mỗi ngày một lần, gom toàn bộ dữ liệu mới trong vòng 24 giờ gần nhất thành một đợt rồi ghi xuống. Cách này phù hợp với bài toán ở đây, khi dữ liệu không đòi hỏi cập nhật tức thời mà chỉ cần được bổ sung đều đặn theo ngày.



Hình 10: Kiến trúc pipeline nạp dữ liệu từ nhiều nguồn. Lịch chạy hằng ngày kích hoạt các chương trình thu thập (crawler); dữ liệu thu được được ghi vào Data Lake trên máy, sắp xếp theo ngày thu thập.

Khác với quy trình **ETL** (gồm ba bước Extract, Transform và Load), nơi dữ liệu được biến đổi nhiều trước khi đưa vào kho phân tích, pipeline trong bài chỉ dừng ở bước nạp: kết nối nguồn, áp dụng một vài quy tắc lọc khi lấy dữ liệu, rồi ghi dữ liệu ở dạng gần với bản gốc.

Pipeline đạt được các mục tiêu sau:

- **Đa nguồn:** Thu thập cùng lúc dữ liệu dạng văn bản (thông tin mô tả bài báo), dữ liệu đa định dạng (video, âm thanh, phụ đề) và dữ liệu hình ảnh.

- **Chỉ lấy phần mới:** Mỗi lần chạy chỉ bổ sung dữ liệu mới trong một khoảng thời gian được cấu hình, không quét lại toàn bộ dữ liệu cũ.
- **Tự động hóa:** Một file khởi chạy duy nhất (`run_ingestion.py`) được lên lịch chạy hằng ngày qua Windows Task Scheduler.

Hướng dẫn lấy API key. Để chạy được pipeline, bạn cần đăng ký API cho hai nguồn YouTube và Unsplash theo các hướng dẫn sau:

- Lấy YouTube Data API v3:
  https://developers.google.com/youtube/v3/getting-started?hl=vi
- Lấy Unsplash API Key:
  https://n1ed.com/doc/generate-unsplash-key

V.1.1. Cấu trúc bài thực hành

Bài thực hành được tổ chức đơn giản: mỗi nguồn dữ liệu nằm trong một thư mục con riêng, cùng một file điều phối đặt ở thư mục gốc. Bảng 1 tóm tắt vai trò của các thành phần chính.

Bảng 1: Cấu trúc bài thực hành và vai trò các file chính.

Đường dẫn	Vai trò
`run_ingestion.py`	File khởi chạy điều phối; nhận tham số `-job` và gọi lần lượt từng chương trình thu thập.
`crawl_arxiv/`	Mã nguồn thu thập thông tin mô tả bài báo từ arXiv.
`crawl_yt_video_audio/`	Mã nguồn thu thập video, âm thanh, phụ đề và thông tin mô tả từ YouTube.
`crawl_unsplash_image/`	Mã nguồn thu thập ảnh và thông tin mô tả từ Unsplash.
`data_lake_local/`	Data Lake trên máy: nơi tập trung toàn bộ dữ liệu đầu ra, chia theo nguồn (`arxiv/`, `youtube/`, `unsplash/`).
`requirements.txt`	Khai báo các thư viện Python dùng chung.
`.env` (từng module)	Chứa khóa API; không đưa lên hệ thống quản lý mã nguồn.

Trong bài thực hành, chính máy tính cá nhân đóng vai trò một Data Lake thu nhỏ: một kho tập trung lưu dữ liệu thô từ cả ba nguồn, để các bước xử lý về sau đọc lại mà không cần can thiệp vào crawler.

Mọi tệp đầu ra được ghi vào thư mục con dạng `YYYY-MM-DD`, thể hiện thời điểm thu thập dữ liệu, không nhất thiết trùng với ngày nguồn công bố. Riêng các file trạng thái (ví dụ `downloaded_ids.json`) được đặt trong thư mục `state/` tách khỏi dữ liệu theo ngày.

V.2. Giới thiệu các nguồn dữ liệu

V.2.1. YouTube

YouTube là kho video phong phú, phù hợp khi cần thu thập bài giảng, video tóm tắt bài báo hoặc thảo luận nghiên cứu AI kèm phụ đề. Trong bài, việc truy cập được thực hiện qua hai lớp:

1. **YouTube Data API v3:** Dùng để tìm video theo từ khóa, lọc theo thời gian đăng và đọc thông tin mô tả (tiêu đề, ngôn ngữ, độ dài).
2. **yt-dlp:** Dùng để tải video, âm thanh, phụ đề và file thông tin mô tả JSON từ đường dẫn video.

Điểm lưu ý: API tìm kiếm **không** có bộ lọc ngôn ngữ chặt; tham số `relevanceLanguage` chỉ ảnh hưởng tới thứ hạng kết quả chứ không loại bỏ video khác ngôn ngữ. Vì vậy, sau bước tìm kiếm, pipeline lọc thêm một lần dựa trên thông tin ngôn ngữ và suy đoán từ tiêu đề.

V.2.2. Unsplash

Unsplash cung cấp ảnh chất lượng cao qua REST API, phù hợp làm tập ảnh minh họa cho các bài toán thị giác máy tính hoặc hệ thống đa định dạng. API cho phép tìm theo từ khóa, sắp xếp theo mức độ mới (`order_by=latest`) và trả về nhiều thông tin mô tả (kích thước, tác giả, đường dẫn).

Theo điều khoản sử dụng API, mỗi khi lưu một ảnh về máy, ứng dụng phải gọi tới địa chỉ `download_location` để Unsplash ghi nhận lượt tải. Pipeline tuân thủ yêu cầu này ngay trong bước nạp.

V.2.3. arXiv

arXiv là kho lưu trữ mở các bài báo khoa học, đặc biệt trong lĩnh vực máy học và thị giác máy tính. API Atom (`export.arxiv.org/api/query`) trả về dữ liệu dạng XML gồm tiêu đề, tác giả, tóm tắt, danh mục, ngày nộp và đường dẫn PDF. Đây là nguồn thuận lợi cho các bài toán gợi ý bài báo, tóm tắt tự động hoặc xây dựng kho văn bản học thuật.

Bảng 2: So sánh ba nguồn dữ liệu trong bài thực hành.

Nguồn	Kiểu dữ liệu	Cách truy cập	Định dạng tệp lưu
YouTube	Video, âm thanh, phụ đề, thông tin mô tả	Data API v3 + yt-dlp	`.mp4`, `.m4a`, `.vtt`, `.info.json`
Unsplash	Hình ảnh + thông tin mô tả	REST API v1	`.jpg` + `metadata.json`
arXiv	Thông tin mô tả bài báo	Atom API (XML)	`papers_YYYY-MM-DD.csv`

V.3. Triển khai bài thực hành

Phần này trình bày logic chính của từng chương trình thu thập dưới dạng mã giả (pseudocode), kèm giải thích các lựa chọn thiết kế. Mã Python đầy đủ nằm trong kho mã của dự án.

V.3.1. Lấy dữ liệu từ YouTube (Video + Audio + Phụ đề)

Cấu hình các quy tắc lọc khi nạp. File `crawl_yt_video_audio/config.yaml` xác định:

- `query`: `"AI Research Papers"` (từ khóa tìm kiếm).
- `days_back`: `1` (khoảng thời gian, tức 24 giờ gần nhất).
- `min_duration_seconds` / `max_duration_seconds`: `120` đến `720` (tức 2 đến 12 phút).
- `require_english`: `true` (chỉ lấy video có thông tin hoặc tiêu đề thể hiện tiếng Anh).
- `exclude_shorts`: `true` (bỏ qua YouTube Shorts).
- `max_results`: số video tối đa mỗi lần chạy.

Cấu trúc thư mục đầu ra.

```
data_lake_local/youtube/
  YYYY-MM-DD/                          # sắp xếp theo ngày thu thập
    video/<video_id>.mp4               # luồng video gốc (yt-dlp)
    audio/<video_id>.m4a               # luồng âm thanh riêng
    subs/<video_id>.en.vtt             # phụ đề tiếng Anh (WebVTT)
    info/<video_id>.info.json          # thông tin mô tả đầy đủ từ yt-dlp
  state/downloaded_ids.json            # file trạng thái: video_id đã lấy
```

Pseudocode cho bước tìm kiếm. Bước này dùng YouTube Data API để lấy danh sách đường dẫn video thỏa các quy tắc lọc.

pse-code-youtube-discovery

```
def get_recent_videos(api_key, skip_ids, config):
    """Tim video moi bang YouTube Data API v3."""
    youtube = build("youtube", "v3", developerKey=api_key)
    published_after = utc_now() - timedelta(days=config.days_back)
    video_urls = []
    deadline = time_now() + config.search_timeout_seconds

    while len(video_urls) < config.max_results and time_now() < deadline:
        # Buoc 1: search.list - tim ung vien theo query va thoi gian
        response = youtube.search().list(
            q=config.query,
            type="video",
            publishedAfter=published_after,
            order="date",
            relevanceLanguage=config.relevance_language,
            maxResults=min(50, config.max_results * 4),
        ).execute()

        candidate_ids = [
            item["id"]["videoId"]
            for item in response["items"]
            if item["id"]["videoId"] not in skip_ids
        ]

        # Buoc 2: videos.list - lay thong tin chi tiet (ngon ngu, do dai)
        details = youtube.videos().list(
            part="snippet,contentDetails",
            id=",".join(candidate_ids),
        ).execute()

        for video_id in candidate_ids:
            item = details[video_id]
            snippet = item["snippet"]
            duration_sec = parse_iso8601(item["contentDetails"]["duration"])

            # Loc bo sung tai nguon
            if config.require_english and not is_likely_english(snippet):
                continue
            if not (config.min_duration <= duration_sec <= config.max_duration):
                continue

            video_urls.append(f"https://www.youtube.com/watch?v={video_id}")
            if len(video_urls) >= config.max_results:
                break

        if not response.get("nextPageToken"):
            break

    return video_urls
```

Pseudocode cho bước tải xuống. Dựa trên danh sách đường dẫn, yt-dlp tải các thành phần và lưu vào thư mục dữ liệu. Trạng thái được cập nhật ngay sau mỗi video, nhờ đó nếu chạy lại, hệ thống không tải trùng.

pse-code-youtube-download

```
def download_assets(urls, base_dir, state):
    """Tai video, audio, phu de va thong tin mo ta; ghi state sau moi item."""
    ensure_dirs(base_dir, ["video", "audio", "info", "subs"])

    for url in urls:
        video_id = extract_video_id(url)
        record = {"url": url, "downloaded_at": utc_now_iso()}

        # Buoc 1: thong tin mo ta + phu de (chua tai media)
        info_path, is_shorts, title = write_info_and_subs(
            url, base_dir / "info", base_dir / "subs"
        )
        if config.exclude_shorts and is_shorts:
            continue

        # Buoc 2: luong video (mp4)
        record["video"] = download_video_stream(url, base_dir / "video")

        # Buoc 3: luong audio (m4a) - tach rieng
        record["audio"] = download_audio_stream(url, base_dir / "audio")

        if record.get("video") or record.get("audio"):
            state[video_id] = record
            save_downloaded_ids(state)   # luu lai trang thai
```

Một số điểm chú ý.

- Không dùng bộ lọc `videoDuration` của API vì cách phân loại quá thô (ngắn dưới 4 phút, trung bình 4 đến 20 phút); thay vào đó lọc chính xác bằng trường `contentDetails.duration`.
- File `downloaded_ids.json` lưu danh sách video đã lấy kèm đường dẫn file; lần chạy sau sẽ bỏ qua các video này.

V.3.2. Lấy dữ liệu từ Unsplash

Các tham số chính được khai báo ở đầu file `crawl_unsplash_image/main.py`:

- `SEARCH_QUERY = "technology"`
- `TODAY_ONLY = True` (chỉ lấy ảnh có `created_at` trong ngày hiện tại).
- `MAX_RESULTS = 20`
- `IMAGE_QUALITY = "regular"`

Khóa API đọc từ file `.env`: biến `ACCESS_KEY`.

Cấu trúc thư mục đầu ra.

```
data_lake_local/unsplash/
  YYYY-MM-DD/<photo_id>.jpg          # ảnh gốc, sắp xếp theo ngày thu thập
  state/downloaded_ids.json          # file trạng thái: photo_id đã lấy
  state/metadata.json                # danh mục thông tin mô tả
```

pse-code-unsplash

```python
def search_photos(query, max_results, skip_ids):
    """Tim anh theo query; chi giu anh publish trong ngay hien tai."""
    photos = []
    target_day = today_local()
    page = 1

    while len(photos) < max_results:
        response = GET("/search/photos", params={
            "query": query, "page": page,
            "per_page": 30, "order_by": "latest",
        })
        results = response["results"]
        if not results:
            break

        page_all_older_than_today = True
        for photo in results:
            created_date = to_local_date(photo["created_at"])

            if created_date >= target_day:
                page_all_older_than_today = False

            if TODAY_ONLY and created_date != target_day:
                continue
            if photo["id"] in skip_ids:
                continue

            photos.append(photo)
            if len(photos) >= max_results:
                break

        # Dung som: order_by=latest, het anh trong ngay -> dung lai
        if TODAY_ONLY and page_all_older_than_today:
            break
        page += 1

    return photos


def download_photos(photos, save_dir, state):
    """Ghi anh va thong tin mo ta vao Data Lake; goi download_location theo quy dinh
                                                API."""
    metadata_index = load_metadata_index()

    for photo in photos:
```

```
        trigger_download_event(photo["links"]["download_location"])

        image_url = photo["urls"][IMAGE_QUALITY]
        file_path = download_image(image_url, save_dir / photo["id"])

        state[photo["id"]] = {
            "title": photo.get("alt_description"),
            "file": file_path,
            "downloaded_at": utc_now_iso(),
        }
        metadata_index[photo["id"]] = build_full_record(photo, file_path)

        save_downloaded_ids(state)
        save_metadata_index(metadata_index)
```

Một số điểm chú ý.

- Cách kết hợp `order_by=latest` với `TODAY_ONLY` và việc dừng sớm giúp giảm số lần gọi API khi đã duyệt hết ảnh trong ngày.
- `metadata.json` đóng vai trò danh mục chi tiết (kích thước, tác giả, đường dẫn) phục vụ tra cứu về sau; `downloaded_ids.json` dùng để kiểm tra trùng nhanh.
- Ảnh được lưu nguyên bản, không qua bước thay đổi kích thước hay tăng cường dữ liệu, đúng tinh thần của việc nạp dữ liệu thô.

V.3.3. Lấy dữ liệu từ arXiv API

Cấu hình. File `crawl_arxiv/config.yaml`:

- `categories`: `[cs.LG, cs.CV]` (các danh mục arXiv cần theo dõi).
- `max_results_per_category`: 50
- `days_back`: 1
- `api.request_delay`: 5 (giây)

Cấu trúc thư mục đầu ra.

```
data_lake_local/arxiv/
  data/papers_YYYY-MM-DD.csv          # thông tin mô tả bài báo (CSV)
  logs/crawl_YYYY-MM-DD.log           # nhật ký từng lần thu thập
```

Các cột của file CSV đầu ra. `arxiv_id`, `title`, `authors`, `abstract`, `categories`, `published`, `updated`, `pdf_url`, `primary_category`.

pse-code-arxiv

```
def build_query(category, days_back):
    """Xay dung truy van Atom API theo category va submittedDate."""
    start = today() - timedelta(days=days_back)
    end = today()
    date_range = f"[{start:%Y%m%d}0000 TO {end:%Y%m%d}2359]"
    return f"cat:{category} AND submittedDate:{date_range}"


def fetch_papers(base_url, category, max_results, days_back):
    """Goi arXiv Atom API va doc feed thanh danh sach dict."""
    query = build_query(category, days_back)
    response = GET(base_url, params={
        "search_query": query,
        "start": 0,
        "max_results": max_results,
        "sortBy": "submittedDate",
        "sortOrder": "descending",
    })
    sleep(request_delay)

    papers = []
    for entry in parse_atom_feed(response):
        papers.append({
            "arxiv_id": extract_arxiv_id(entry),
            "title": normalize_whitespace(entry.title),
            "authors": join_author_names(entry.authors),
            "abstract": normalize_whitespace(entry.summary),
            "categories": join_category_tags(entry.tags),
            "published": entry.published,
            "updated": entry.updated,
            "pdf_url": find_pdf_link(entry.links),
            "primary_category": category,
        })
    return papers


def main():
    config = load_config("config.yaml")
    setup_logging(config.log_dir)

    all_papers = []
    for category in config.categories:
        try:
            batch = fetch_papers(
                config.api.base_url, category,
                config.max_results_per_category, config.days_back,
            )
            all_papers.extend(batch)
        except RequestException as e:
            log_error(category, e)
            continue

```

```
    unique = deduplicate_by_arxiv_id(all_papers)
    save_to_csv(unique, config.output_dir)  # gop neu file cung ngay da ton tai
```

Một số điểm chú ý.

- Một bài báo có thể nằm trong nhiều danh mục; hàm `deduplicate_by_arxiv_id` loại bỏ bản trùng trước khi ghi file.
- Hàm `save_to_csv` gộp với file CSV cùng ngày nếu đã có, nhờ vậy có thể chạy lại trong ngày mà không mất dữ liệu.
- Nhật ký được ghi vào `logs/crawl_YYYY-MM-DD.log`, giúp kiểm tra lại từng lần chạy, rất hữu ích khi chạy tự động qua Task Scheduler.

V.4. Tạo lịch lấy dữ liệu hàng ngày

Để pipeline chạy tự động, ta tách rõ hai phần: *điều phối* (gọi các chương trình thu thập theo đúng thứ tự) và *lên lịch* (quyết định khi nào chạy).

V.4.1. Điều phối pipeline với `run_ingestion.py`

File khởi chạy nhận tham số dòng lệnh `-job` với các giá trị `arxiv`, `yt`, `unsplash` hoặc `all`. Khi chọn `all`, ba chương trình thu thập chạy lần lượt theo thứ tự arXiv, YouTube, Unsplash. Mỗi chương trình chạy trong một tiến trình con riêng với `sys.executable`, đảm bảo dùng đúng môi trường Python khi được Task Scheduler kích hoạt.

pse-code-orchestrator

```
REPO_ROOT = directory_of(__file__)

JOB_SCRIPTS = {
    "arxiv":    "crawl_arxiv/crawl_arxiv.py",
    "yt":       "crawl_yt_video_audio/main.py",
    "unsplash": "crawl_unsplash_image/main.py",
}

def run_python(rel_path):
    script = REPO_ROOT / rel_path
    proc = subprocess.run(
        [sys.executable, str(script)],
        cwd=str(REPO_ROOT),
    )
    return proc.returncode

def main(argv):
    jobs = parse_job_argument(argv)  # --job all | arxiv | yt | unsplash
    if "all" in jobs:
        jobs = ["arxiv", "yt", "unsplash"]

```

```
    exit_code = 0
    for job in jobs:
        rc = run_python(JOB_SCRIPTS[job])
        if rc != 0 and exit_code == 0:
            exit_code = rc
    return exit_code
```

Thiết kế này giúp scheduler chỉ cần gọi một lệnh duy nhất (`-job all`), đồng thời vẫn cho phép chạy riêng từng nguồn khi cần kiểm tra lỗi.

V.4.2. Đăng ký Windows Task Scheduler

Trên Windows, Task Scheduler đóng vai trò giống cron trên Linux. Nên trỏ thẳng tới *đường dẫn tuyệt đối* của Python trong môi trường conda, thay vì gọi `conda activate` bên trong tác vụ đã lên lịch.

pse-code-scheduler

```
# PowerShell (chay voi quyen Administrator)
$python  = "<DUONG_DAN_DEN>\python.exe"   # vi du: python.exe trong moi truong conda
$workDir = "<DUONG_DAN_DEN>\crawl_project" # thu muc goc cua du an

$action = New-ScheduledTaskAction `
    -Execute $python `
    -Argument "run_ingestion.py --job all" `
    -WorkingDirectory $workDir

$trigger = New-ScheduledTaskTrigger -Daily -At 8:00AM

$settings = New-ScheduledTaskSettingsSet `
    -StartWhenAvailable -DontStopOnIdleEnd

Register-ScheduledTask `
    -TaskName "Crawl Papers Daily" `
    -Action $action -Trigger $trigger -Settings $settings `
    -Description "Ingest arxiv + youtube + unsplash daily"
```

V.4.3. Kết luận phần thực hành

Bài thực hành đã xây dựng một pipeline nạp dữ liệu hoàn chỉnh với ba đặc điểm chính: (i) đa nguồn và đa định dạng, (ii) chỉ lấy phần dữ liệu mới và có kiểm soát ngay tại nguồn, (iii) tự động hóa qua scheduler. Toàn bộ dữ liệu thu được, từ video, âm thanh, phụ đề, hình ảnh cho tới thông tin mô tả bài báo, đều được ghi ở dạng thô vào thư mục phân theo ngày trên máy local. Cấu trúc này chính là một data lake thu nhỏ: nơi tập trung dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc trong cùng một kho, giữ nguyên định dạng gốc mà chưa áp đặt lược đồ xử lý nào. Đây là nền tảng để mở rộng sang các bước tiếp theo trong vòng đời dữ liệu, từ lưu trữ thô tới xử lý, lập chỉ mục và triển khai ứng dụng.

VI. Tài liệu tham khảo

- https://www.databricks.com/blog/what-is-data-ingestion
- https://insightsoftware.com/encyclopedia/data-source-types/
- https://www.ibm.com/think/topics/data-ingestion
- https://www.ibm.com/think/topics/data-lake

- Hết -